Event Horizon First PR Report

Generated from the repository report source, current screenshots, tests, and PR diff.

Executive Summary

Event Horizon now has a first playable vertical slice on `feat/first-playable`: a mobile-first Vite + PixiJS v8 canvas game where the player taps, swipes, and captures dark-energy orbs to delay a black-hole collapse. The run is deterministic from seed plus input timings, uses a fixed 60 Hz simulation step, records replay payloads, exports a three-frame share poster, and includes a minimal score-submit endpoint. The repo builds locally, unit tests pass, mobile Chrome smoke tests pass, and screenshots were captured from the actual running build.

Assumptions

- The repo was greenfield except for the initial commit, so a plain static Vite scaffold was chosen.
- The target playable slice is a prototype, not a balanced production game.
- `2026-05-29` is used as the alpha seed date because it was provided in the task context.
- GitHub Pages is the public static target at `/event-horizon/`; Netlify is supported with a root base path.
- A custom deterministic simulation is sufficient for radial gravity, tether capture, flybys, and shadow arms; no Matter.js or Planck.js dependency is justified yet.
- The Netlify endpoint validates and acknowledges replay payloads only; storage and anti-cheat are backlog items.

Chosen Stack and Why

- Vite + TypeScript: smallest viable static scaffold, fast local dev, and straightforward GitHub Pages base-path support.
- PixiJS v8: direct WebGL-preferred 2D rendering with a scene graph and generated textures. The implementation follows the PixiJS v8 async `Application.init()` pattern and `preference: 'webgl'` option from the official PixiJS docs.
- Custom simulation: deterministic, low-allocation motion was simpler than integrating a physics engine for this slice.
- Vitest: fast deterministic simulation and endpoint unit tests.
- Playwright with Chrome: local mobile/touch smoke checks for the target browser.
- Netlify Functions: minimal serverless score endpoint in-repo, matching Netlify's default `/.netlify/functions/<name>` routing.
- Google Apps Script sample: alternate deploy path using `doPost` and `ContentService` JSON output.

Sources used: PixiJS Application docs, PixiJS official skills repo, Netlify Functions docs, Google Apps Script Content Service docs, and Vite base config docs.

Implementation Plan and Estimated Hours

1. Repo inspection and AGENTS.md: 0.25h
2. Vite + TypeScript + PixiJS scaffold: 0.75h
3. Deterministic simulation, fixed-step loop, RNG, and replay payloads: 2.0h
4. PixiJS scene, galaxy background, black hole, orbs, tethers, flybys, shadow arms, HUD: 2.0h
5. Tap/swipe input handler and logical viewport scaling: 1.0h
6. Collapse animation and posterizer export: 1.25h
7. Netlify score function and GAS sample: 0.75h
8. Unit, e2e, score endpoint, and artifact capture tests: 1.25h
9. README, PR body, PDF report, and final polish: 1.25h

Total estimate: 10.5h

Prioritized Backlog

1. Add durable score storage and abuse limits for Netlify or GAS.
2. Add replay playback UI and a deterministic replay verifier in CI.
3. Balance phase thresholds, orb spawn curves, and capture scoring through playtest data.
4. Add visual capture streak feedback and better end-run poster composition.
5. Add sound, haptics, and reduced-motion options.
6. Add GitHub Actions for build, lint, unit tests, and Playwright smoke tests.

7. Add accessibility affordances for non-touch desktop play.

Git and PR Commands

```
git switch -c feat/first-playable
npm install
npm run build
npm run lint
npm run test
npm run test:e2e
npm run score:test
npm run capture:artifacts
npm run report:pdf
git add .
git commit -m "Build first playable Event Horizon slice"
git push -u origin feat/first-playable
gh pr create --base main --head feat/first-playable --title "Build first playable Event Horizon slice" --body-file docs/pr-body.md
```

File Tree

```
AGENTS.md
README.md
docs/artifacts/collapse-mobile.jpg
docs/artifacts/gameplay-mobile.jpg
docs/artifacts/share-poster.jpg
docs/first-pr-report.md
docs/pr-body.md
eslint.config.js
gas/score-submit.gs
index.html
netlify.toml
netlify/functions/score-submit.mjs
package-lock.json
package.json
playwright.config.ts
scripts/capture-artifacts.mjs
scripts/generate-report-pdf.mjs
scripts/test-score-submit.mjs
src/game/EventHorizonGame.ts
src/game/FixedStepLoop.ts
src/game/InputHandler.ts
src/game/Simulation.ts
src/game/constants.ts
src/game/math.ts
src/game/posterizer.ts
src/game/rng.ts
src/game/scoreClient.ts
src/game/types.ts
src/main.ts
src/styles.css
src/vite-env.d.ts
tests/e2e/playable.spec.ts
tests/mjs.d.ts
tests/score-submit.test.ts
tests/simulation.test.ts
tsconfig.json
vite.config.ts
```

Changed and Created File Notes

- `AGENTS.md`: practical repo instructions for future Codex work.
- `.gitignore`: ignores local OS, environment, build, coverage, and dependency artifacts.
- `README.md`: project overview, commands, assumptions, deployment, replay payload, and PR workflow.
- `index.html`: minimal app shell with canvas mount and compact restart/share controls.
- `src/main.ts`: bootstraps the game and exposes a small debug/test API.
- `src/styles.css`: fullscreen mobile-first canvas shell and compact icon controls.
- `src/game/EventHorizonGame.ts`: PixiJS scene, rendering, HUD, scaling, poster export, score submit call.
- `src/game/Simulation.ts`: deterministic gameplay state, phase changes, orbs, flybys, shadow arms, replay events.
- `src/game/FixedStepLoop.ts`: fixed 60 Hz simulation loop decoupled from rendering.
- `src/game/InputHandler.ts`: pointer/touch tap and swipe mapping to logical world coordinates.
- `src/game/rng.ts`: string hash and `mulberry32` seeded RNG.
- `src/game/posterizer.ts`: creates vertical share image from three gameplay frames.
- `src/game/scoreClient.ts`: posts replay payloads to the score endpoint.
- `src/game/constants.ts`, `math.ts`, `types.ts`, `vite-env.d.ts`: shared constants, helpers, and types.
- `netlify/functions/score-submit.mjs`: minimal score validation endpoint.
- `gas/score-submit.gs`: Google Apps Script `doPost` JSON endpoint sample.
- `netlify.toml`: Netlify build, publish, and functions directory config.
- `package.json`, `package-lock.json`: dependencies and runnable scripts.
- `tsconfig.json`, `vite.config.ts`, `eslint.config.js`, `playwright.config.ts`: TypeScript, Vite, lint, and browser test config.
- `tests/**/*.ts`: deterministic replay, score endpoint, and mobile Chrome smoke tests.

- `scripts/capture-artifacts.mjs`: captures mobile gameplay, poster, and collapse images.
- `scripts/test-score-submit.mjs`: standalone endpoint sanity test.
- `scripts/generate-report-pdf.mjs`: generates the requested PDF artifact.
- `docs/artifacts/*`: compressed screenshots and poster image from the actual build.
- `docs/pr-body.md`: PR description source.

Major Code Snippets

`index.html`

```
<main id="app" aria-label="Event Horizon game">
  <div id="game-shell">
    <div id="game-root" aria-label="Playable canvas"></div>
    <div id="status-layer" aria-live="polite">
      <button id="restart-button" type="button" aria-label="Restart run" title="Restart run">↻</button>
      <button id="share-button" type="button" aria-label="Create share poster" title="Create share poster">⇪</button>
      <a id="poster-link" download="event-horizon-poster.png" aria-label="Download share poster">⇩</a>
    </div>
  </div>
</main>
<script type="module" src="/src/main.ts"></script>
```

Main Entry

```
const game = new EventHorizonGame(root, {
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000
});

await game.start();
```

Fixed-Step Loop

```
while (this.accumulatorMs >= FIXED_STEP_MS) {
  this.step(FIXED_STEP_MS);
  this.accumulatorMs -= FIXED_STEP_MS;
}

this.render(this.accumulatorMs / FIXED_STEP_MS);
```

`mulberry32`

```
export function mulberry32(seed: number): RandomSource {
  let state = seed >>> 0;
  return () => {
    state = (state + 0x6d2b79f5) >>> 0;
    let next = state;
    next = Math.imul(next ^ (next >>> 15), next | 1);
    next ^= next + Math.imul(next ^ (next >>> 7), next | 61);
    return ((next ^ (next >>> 14)) >>> 0) / 4294967296;
  };
}
```

PixiJS Init

```
await this.app.init({
  autoDensity: true,
  autoStart: false,
  backgroundAlpha: 0,
  preference: 'webgl',
  preserveDrawingBuffer: true,
  powerPreference: 'high-performance',
  resizeTo: this.root,
  resolution: Math.min(window.devicePixelRatio || 1, 2)
});
```

Input Handler

```
if (moved >= this.swipeThresholdSq) {
  this.callbacks.onSwipe(this.start, end);
} else {
  this.callbacks.onTap(end);
}
```

Posterizer

```
const loadedFrames = await Promise.all(frames.slice(-3).map((frame) => loadImage(frame.dataUrl)));
context.fillText(`survived ${formatTime(stats.survivalMs)}  •  phase ${stats.phase}`, 52, footerY + 154);
return canvas.toDataURL('image/png', 0.86);
```

GAS `doPost`

```
function doPost(e) {
  var payload = JSON.parse(e.postData.contents || '{}');
  return ContentService.createTextOutput(JSON.stringify({
    ok: true,
    acceptedAt: new Date().toISOString(),
    score: Number(payload.score || 0),
    survivalMs: Number(payload.survivalMs || 0)
  })).setMimeType(ContentService.MimeType.JSON);
}
```

Sample Replay Payload

```
{
  "version": 1,
  "seed": "eh-2026-05-29-alpha",
  "startedAt": 1780051200000,
  "survivalMs": 67421,
  "score": 1280,
  "energyCaptured": 87,
  "maxStreak": 24,
  "tapEvents": [],
  "swipeEvents": [],
  "phaseTransitions": []
}
```

Test Results

- `npm run build`: passed. TypeScript compiled and Vite produced `dist/`.
- `npm run lint`: passed.
- `npm run test`: passed, 2 files and 5 tests.
- `npm run test:e2e`: passed, 4 mobile Chrome tests.
- `npm run score:test`: passed, score endpoint returned status `200` and `{ "ok": true }`.
- Local Chrome smoke test: passed via Playwright and in-app browser visual inspection.
- Deterministic replay sanity: passed. Same seed plus same input timings produced identical replay payloads.
- Touch simulation sanity: passed. Tap and swipe events are captured in replay payloads.
- Posterizer export: passed. Browser test returns a `...` poster.
- Score submit test: passed against `/.netlify/functions/score-submit` handler import.

Performance Notes

- The simulation uses a fixed 60 Hz step and avoids allocating per-frame input or entity arrays.
- Orb and flyby sprites use generated textures rather than drawing new art every frame.
- Background art is drawn once; per-frame `Graphics` clearing is limited to tethers, arms, HUD fill, and flyby trails.
- No heavy filters or physics engine are used.
- `preserveDrawingBuffer` supports poster screenshots but can cost some GPU performance; if poster export changes later, replacing it with targeted render-texture capture may be better.
- The main visible bottleneck risk is `canvas.toDataURL()` during poster capture; it is user-triggered rather than hot-loop work.

Deployment Notes

Netlify

- Build command: `VITE_BASE_PATH=/ npm run build`
- Publish directory: `dist`
- Functions directory: `netlify/functions`
- Score function route: `/.netlify/functions/score-submit`
- Branch deploys: connect the GitHub repo in Netlify and enable branch deploys for PR previews.
- Local Netlify dev: `npx netlify dev` from the repo root.

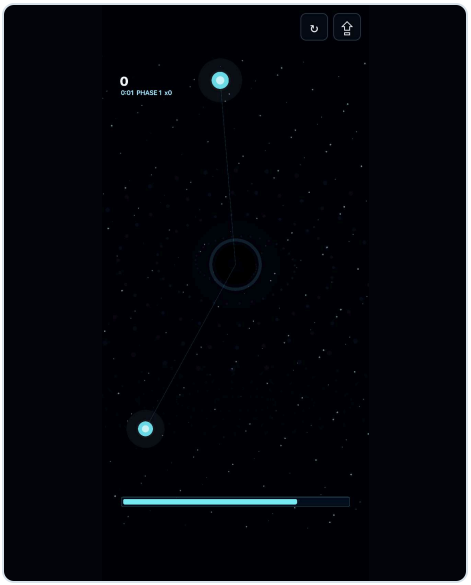
GitHub Pages

- Default Vite `base` is `/event-horizon/`.
- Static build command: `npm run build`
- Publish the generated `dist/` content through the chosen GitHub Pages workflow.
- Environment assumption: GitHub Pages uses a subpath, while Netlify uses root. Override with `VITE_BASE_PATH=/` for root deploys.

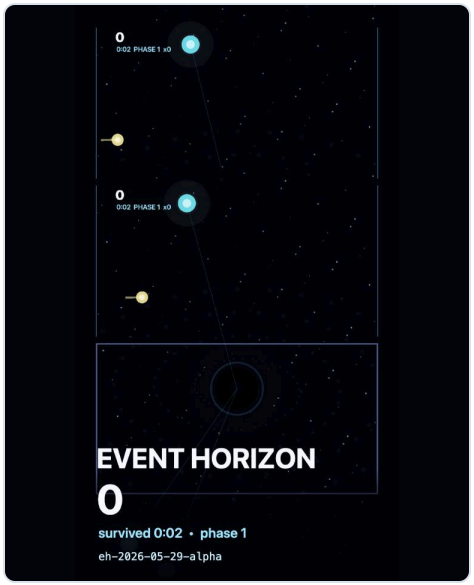
PR Summary

This PR introduces the first playable Event Horizon slice: deterministic simulation, PixiJS rendering, one-thumb input, replay payloads, phase-based collapse pressure, score submission samples, tests, documentation, screenshots, and this PDF report.

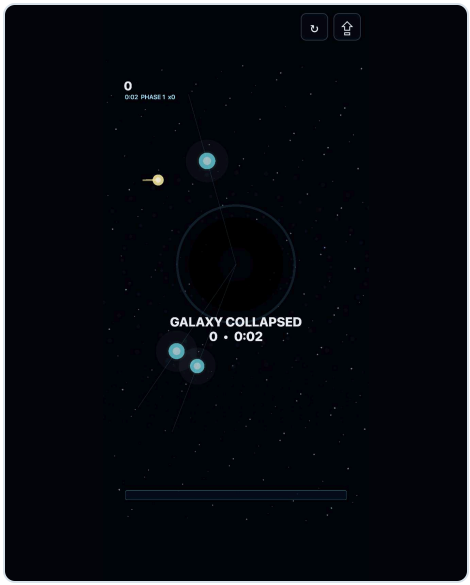
Screenshots and Poster



Gameplay mobile screenshot



Three-frame share poster



Collapse mobile screenshot

Changed Files From origin/main

```
A       .gitignore
A       AGENTS.md
A       README.md
A       docs/artifacts/collapse-mobile.jpg
A       docs/artifacts/gameplay-mobile.jpg
A       docs/artifacts/share-poster.jpg
A       docs/first-pr-report.md
A       docs/first-pr-report.pdf
A       docs/pr-body.md
A       eslint.config.js
A       gas/score-submit.gs
A       index.html
A       netlify.toml
A       netlify/functions/score-submit.mjs
A       package-lock.json
A       package.json
A       playwright.config.ts
A       scripts/capture-artifacts.mjs
A       scripts/generate-report-pdf.mjs
A       scripts/test-score-submit.mjs
A       src/game/EventHorizonGame.ts
A       src/game/FixedStepLoop.ts
A       src/game/InputHandler.ts
A       src/game/Simulation.ts
A       src/game/constants.ts
A       src/game/math.ts
A       src/game/posterizer.ts
A       src/game/rng.ts
A       src/game/scoreClient.ts
A       src/game/types.ts
A       src/main.ts
A       src/styles.css
A       src/vite-env.d.ts
A       tests/e2e/playable.spec.ts
A       tests/mjs.d.ts
A       tests/score-submit.test.ts
A       tests/simulation.test.ts
A       tsconfig.json
A       vite.config.ts


M       docs/artifacts/collapse-mobile.jpg
M       docs/artifacts/gameplay-mobile.jpg
M       docs/artifacts/share-poster.jpg
M       docs/first-pr-report.pdf
M       package-lock.json
M       package.json
M       scripts/capture-artifacts.mjs
M       scripts/generate-report-pdf.mjs
```

Diff Stat

```
 .gitignore                        |    9 +
 AGENTS.md                         |    9 +
 README.md                         |  105 +
 docs/artifacts/collapse-mobile.jpg |  Bin 0 -> 2601 bytes
 docs/artifacts/gameplay-mobile.jpg |  Bin 0 -> 2119 bytes
 docs/artifacts/share-poster.jpg    |  Bin 0 -> 2815 bytes
```

```
 docs/first-pr-report.md            |  299 +++
 docs/first-pr-report.pdf           |  Bin 0 -> 18914 bytes
 docs/pr-body.md                    |   19 +
 eslint.config.js                   |   39 +
 gas/score-submit.gs                |   29 +
 index.html                         |   23 +
 netlify.toml                       |   12 +
 netlify/functions/score-submit.mjs |   67 +
 package-lock.json                  | 3970 ++++++++++++++++++++++++++++++++++++++
 package.json                       |   30 +
 playwright.config.ts               |   31 +
 scripts/capture-artifacts.mjs      |   72 +
 scripts/generate-report-pdf.mjs    |  181 ++
 scripts/test-score-submit.mjs      |   30 +
 src/game/EventHorizonGame.ts       |  475 +++++
 src/game/FixedStepLoop.ts          |   54 +
 src/game/InputHandler.ts           |   79 +
 src/game/Simulation.ts             |  553 +++++
 src/game/constants.ts              |   11 +
 src/game/math.ts                   |   42 +
 src/game/posterizer.ts             |  110 +
 src/game/rng.ts                    |   25 +
 src/game/scoreClient.ts            |   35 +
 src/game/types.ts                  |   90 +
 src/main.ts                        |   48 +
 src/styles.css                     |   92 +
 src/vite-env.d.ts                  |    9 +
 tests/e2e/playable.spec.ts         |   48 +
 tests/mjs.d.ts                     |    4 +
 tests/score-submit.test.ts         |   40 +
 tests/simulation.test.ts           |   41 +
 tsconfig.json                      |   20 +
 vite.config.ts                     |   21 +
 39 files changed, 6722 insertions(+)
```

Representative Diff Excerpt

```
diff --git a/src/game/EventHorizonGame.ts b/src/game/EventHorizonGame.ts
new file mode 100644
index 0000000..61fdc93
--- /dev/null
+++ b/src/game/EventHorizonGame.ts
@@ -0,0 +1,475 @@
+import {
+  Application,
+  Container,
+  Graphics,
+  Sprite,
+  Text,
+  TextStyle,
+  type Texture,
+  type Renderer
+} from 'pixi.js';
+import {
+  BLACK_HOLE_X,
+  BLACK_HOLE_Y,
+  MAX_ENERGY,
+  WORLD_HEIGHT,
+  WORLD_WIDTH
+} from './constants';
+import { FixedStepLoop } from './FixedStepLoop';
+import { InputHandler, type WorldPoint } from './InputHandler';
+import { clamp, formatTime } from './math';
+import { createSharePoster, type PosterFrame } from './posterizer';
+import { submitScore } from './scoreClient';
+import { Simulation, type SimulationOptions } from './Simulation';
+import type { FlybyState, OrbState, ReplayPayload, ShadowArmState, SimulationSnapshot } from './types';
+
+interface OrbView {
+  sprite: Sprite;
+  glow: Sprite;
+}
+
+interface FlybyView {
+  sprite: Sprite;
+  streak: Graphics;
+}
+
+export class EventHorizonGame {
+  private readonly app = new Application<Renderer>();
+  private readonly world = new Container();
+  private readonly background = new Graphics();
+  private readonly tetherLayer = new Graphics();
+  private readonly armLayer = new Graphics();
+  private readonly blackHole = new Graphics();
+  private readonly collapseLayer = new Graphics();
+  private readonly hud = new Container();
+  private readonly energyBarBack = new Graphics();
+  private readonly energyBarFill = new Graphics();
+  private readonly scoreText = new Text({
+    text: '0',
+    style: new TextStyle({
+      fill: '#f7fbff',
+      fontFamily: 'Inter, system-ui, sans-serif',
+      fontSize: 52,
+      fontWeight: '800'
+    })
+  });
+  private readonly metaText = new Text({
+    text: '',
```

```
+    style: new TextStyle({
+      fill: '#9fe7ff',
+      fontFamily: 'Inter, system-ui, sans-serif',
+      fontSize: 26,
+      fontWeight: '600'
+    })
+  });
+  private readonly endText = new Text({
+    text: '',
+    style: new TextStyle({
+      align: 'center',
+      fill: '#f7fbff',
+      fontFamily: 'Inter, system-ui, sans-serif',
+      fontSize: 54,
+      fontWeight: '800'
+    })
+  });
+  private readonly sim: Simulation;
+  private readonly loop: FixedStepLoop;
+  private input?: InputHandler;
+  private orbViews: OrbView[] = [];
+  private flybyViews: FlybyView[] = [];
+  private frameSamples: PosterFrame[] = [];
+  private sampleCooldownMs = 0;
+  private scale = 1;
+  private offsetX = 0;
+  private offsetY = 0;
+  private scoreSubmitted = false;
+
+  constructor(
+    private readonly root: HTMLElement,
+    options: SimulationOptions
+  ) {
+    this.sim = new Simulation(options);
+    this.loop = new FixedStepLoop(
+      (dtMs) => this.step(dtMs),
+      () => this.render()
+    );
+  }
+
+  async start(): Promise<void> {
+    await this.app.init({
+      autoDensity: true,
+      autoStart: false,
+      backgroundAlpha: 0,
+      clearBeforeRender: true,
+      hello: false,
+      preference: 'webgl',
+      preserveDrawingBuffer: true,
+      powerPreference: 'high-performance',
+      resizeTo: this.root,
+      resolution: Math.min(window.devicePixelRatio || 1, 2)
+    });
+
+    this.root.appendChild(this.app.canvas);
+    this.buildScene();
+    this.input = new InputHandler(this.app.canvas, {
+      screenToWorld: (clientX, clientY) => this.screenToWorld(clientX, clientY),
+      onTap: (point) => this.sim.applyTap(point.x, point.y),
+      onSwipe: (start, end) => this.sim.applySwipe(start.x, start.y, end.x, end.y)
+    });
+    window.addEventListener('resize', this.resize);
+    this.resize();
+    this.loop.start();
+  }
+
+  restart(): void {
+    this.sim.reset();
+    this.scoreSubmitted = false;
+    this.frameSamples = [];
+    this.sampleCooldownMs = 0;
+    this.loop.resetClock();
+  }
+
+  forceEnd(): void {
+    this.sim.forceEnd();
+  }
+
+  destroy(): void {
+    this.loop.stop();
+    this.input?.destroy();
+    window.removeEventListener('resize', this.resize);
+    this.app.destroy(true, { children: true, texture: true });
+  }
+
+  getReplayPayload(): ReplayPayload {
+    return this.sim.getReplayPayload();
+  }
+
+  getSnapshot(): SimulationSnapshot {
+    return this.sim.getSnapshot();
+  }
+
+  async exportPoster(): Promise<string> {
+    this.sampleFrame('current');
+    const snapshot = this.sim.getSnapshot();
+    const frames = this.frameSamples.length >= 3 ? this.frameSamples : this.makeFallbackFrames();
+    return createSharePoster(frames, {
+      score: snapshot.score,
+      survivalMs: this.sim.getReplayPayload().survivalMs,
+      seed: this.sim.seed,
+      phase: snapshot.phase
```

```
+    });
+  }
+
+  private buildScene(): void {
+    this.world.sortableChildren = false;
+    this.app.stage.addChild(this.world);
+    this.world.addChild(this.background);
+    this.world.addChild(this.armLayer);
+    this.world.addChild(this.blackHole);
+    this.world.addChild(this.tetherLayer);
+    this.createOrbViews();
+    this.createFlybyViews();
+    this.world.addChild(this.hud);
+    this.hud.addChild(this.energyBarBack, this.energyBarFill, this.scoreText, this.metaText, this.endText);
+    this.world.addChild(this.collapseLayer);
+    this.drawBackground();
+    this.drawHudBack();
+    this.endText.anchor.set(0.5);
+    this.endText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.58);
+  }
+
+  private createOrbViews(): void {
+    const orbTexture = this.makeOrbTexture(0x78f2ff, 32, false);
+    const glowTexture = this.makeOrbTexture(0xbaf8ff, 76, true);
+    for (let index = 0; index < 24; index += 1) {
+      const glow = new Sprite(glowTexture);
+      glow.anchor.set(0.5);
+      glow.blendMode = 'add';
+      glow.visible = false;
+      const sprite = new Sprite(orbTexture);
+      sprite.anchor.set(0.5);
+      sprite.visible = false;
+      this.world.addChild(glow, sprite);
+      this.orbViews.push({ sprite, glow });
+    }
+  }
+
+  private createFlybyViews(): void {
+    const texture = this.makeOrbTexture(0xfff0a3, 24, false);
+    for (let index = 0; index < 6; index += 1) {
+      const streak = new Graphics();
+      const sprite = new Sprite(texture);
+      sprite.anchor.set(0.5);
+      streak.visible = false;
+      sprite.visible = false;
+      this.world.addChild(streak, sprite);
+      this.flybyViews.push({ sprite, streak });
+    }
+  }
+
+  private makeOrbTexture(color: number, radius: number, soft: boolean): Texture {
+    const graphics = new Graphics();
+    const alpha = soft ? 0.14 : 0.9;
+    graphics.circle(radius, radius, radius).fill({ color, alpha });
+    graphics.circle(radius, radius, radius * 0.48).fill({ color: 0xffffff, alpha: soft ? 0.12 : 0.65 });
+    return this.app.renderer.generateTexture(graphics);
+  }
+
+  private drawBackground(): void {
+    this.background.clear();
+    this.background.rect(0, 0, WORLD_WIDTH, WORLD_HEIGHT).fill(0x03040a);
+
+    for (let index = 0; index < 240; index += 1) {
+      const x = (index * 197.63) % WORLD_WIDTH;
+      const y = (index * 311.19) % WORLD_HEIGHT;
+      const depth = (index % 11) / 10;
+      const alpha = 0.18 + depth * 0.38;
+      const size = 1.1 + depth * 2.2;
+      this.background.circle(x, y, size).fill({ color: 0xbfeaff, alpha });
+    }
+
+    for (let index = 0; index < 150; index += 1) {
+      const t = index / 149;
+      const angle = t * Math.PI * 8.8;
+      const radius = 82 + t * 690;
+      const x = BLACK_HOLE_X + Math.cos(angle) * radius;
+      const y = BLACK_HOLE_Y + Math.sin(angle) * radius * 0.57;
+      const color = index % 2 === 0 ? 0x305da6 : 0x6e366f;
+      this.background.circle(x, y, 3 + t * 7).fill({ color, alpha: 0.07 + (1 - t) * 0.08 });
+      this.background.circle(WORLD_WIDTH - x, WORLD_HEIGHT * 0.88 - y * 0.34, 2 + t * 4).fill({
+        color: 0x1aaf96,
+        alpha: 0.035
+      });
+    }
+  }
+
+  private drawHudBack(): void {
+    this.energyBarBack.clear();
+    this.energyBarBack.roundRect(78, WORLD_HEIGHT - 132, WORLD_WIDTH - 156, 38, 7).fill({
+      color: 0x061120,
+      alpha: 0.86
+    });
+    this.energyBarBack.roundRect(78, WORLD_HEIGHT - 132, WORLD_WIDTH - 156, 38, 7).stroke({
+      color: 0x78f2ff,
+      alpha: 0.36,
+      width: 2
+    });
+    this.scoreText.position.set(72, 70);
+    this.metaText.position.set(76, 132);
+  }
+
+  private step(dtMs: number): void {
```

```
+    const wasEnded = this.sim.getSnapshot().ended;
+    this.sim.step(dtMs);
+    const snapshot = this.sim.getSnapshot();
+    this.sampleCooldownMs -= dtMs;
+    if (this.sampleCooldownMs <= 0 && !snapshot.ended) {
+      this.sampleFrame(`t${Math.round(snapshot.timeMs)}`);
+      this.sampleCooldownMs = 1400;
+    }
+    if (!wasEnded && snapshot.ended && !this.scoreSubmitted) {
+      this.scoreSubmitted = true;
+      void submitScore(this.sim.getReplayPayload());
+    }
+  }
+
+  private render(): void {
+    const snapshot = this.sim.getSnapshot();
+    this.world.position.set(this.offsetX, this.offsetY);
+    this.world.scale.set(this.scale);
+    this.renderShadowArms(snapshot.shadowArms, snapshot.phase);
+    this.renderBlackHole(snapshot);
+    this.renderTethers(snapshot.orbs);
+    this.renderOrbs(snapshot.orbs, snapshot.collapseT);
+    this.renderFlybys(snapshot.flybys);
+    this.renderHud(snapshot);
+    this.renderCollapse(snapshot);
+    this.app.render();
+  }
+
+  private renderShadowArms(arms: readonly ShadowArmState[], phase: number): void {
+    this.armLayer.clear();
+    if (phase < 2) {
+      return;
+    }
+    for (const arm of arms) {
+      const length = 420 + phase * 90;
+      const endX = BLACK_HOLE_X + Math.cos(arm.angle) * length;
+      const endY = BLACK_HOLE_Y + Math.sin(arm.angle) * length;
+      const alpha = arm.stunMs > 0 ? 0.12 : 0.12 + arm.intensity * 0.32;
+      this.armLayer.moveTo(BLACK_HOLE_X, BLACK_HOLE_Y);
+      this.armLayer.lineTo(endX, endY);
+      this.armLayer.stroke({ color: arm.stunMs > 0 ? 0x8ff9ff : 0x1b102b, alpha, width: 50 + phase * 7 });
+      this.armLayer.moveTo(BLACK_HOLE_X, BLACK_HOLE_Y);
+      this.armLayer.lineTo(endX, endY);
+      this.armLayer.stroke({ color: 0x8d62ff, alpha: alpha * 0.45, width: 8 });
+    }
+  }
+
+  private renderBlackHole(snapshot: SimulationSnapshot): void {
+    this.blackHole.clear();
+    const phaseT = (snapshot.phase - 1) / 3;
+    const pulse = Math.sin(snapshot.timeMs * 0.004) * 0.5 + 0.5;
+    const radius = 80 + phaseT * 54 + snapshot.collapseT * 260;
+    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 2.2).fill({
+      color: 0x0a1424,
+      alpha: 0.08 + phaseT * 0.13
+    });
+    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 1.25).stroke({
+      color: 0x6bcfff,
+      alpha: 0.14 + phaseT * 0.26,
+      width: 7 + pulse * 6
+    });
+    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius).fill({ color: 0x000000, alpha: 0.82 });
+    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 0.38).fill({ color: 0x03040a, alpha: 1 });
+  }
+
+  private renderTethers(orbs: readonly OrbState[]): void {
+    this.tetherLayer.clear();
+    for (const orb of orbs) {
+      if (!orb.active) {
+        continue;
+      }
+      const alpha = orb.captured ? 0.72 : orb.frozenMs > 0 ? 0.5 : 0.19;
+      this.tetherLayer.moveTo(BLACK_HOLE_X, BLACK_HOLE_Y);
+      this.tetherLayer.lineTo(orb.x, orb.y);
+      this.tetherLayer.stroke({
+        color: orb.captured ? 0xd9fbff : 0x5bc7ff,
+        alpha,
+        width: orb.captured ? 6 : orb.frozenMs > 0 ? 4 : 2
+      });
+    }
+  }
+
+  private renderOrbs(orbs: readonly OrbState[], collapseT: number): void {
+    for (let index = 0; index < this.orbViews.length; index += 1) {
+      const orb = orbs[index];
+      const view = this.orbViews[index];
+      if (!orb?.active) {
+        view.sprite.visible = false;
+        view.glow.visible = false;
+        continue;
+      }
+      const bendX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * (1 - collapseT * 0.88);
+      const bendY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * (1 - collapseT * 0.88);
+      const frozen = orb.frozenMs > 0;
+      view.sprite.visible = true;
+      view.glow.visible = true;
+      view.sprite.position.set(bendX, bendY);
+      view.glow.position.set(bendX, bendY);
+      const scale = orb.radius / 32;
+      view.sprite.scale.set(scale * (orb.captured ? 1.35 : frozen ? 1.22 : 1));
+      view.glow.scale.set(scale * (orb.captured ? 1.9 : frozen ? 1.45 : 1.08));
+      view.glow.alpha = orb.captured ? 1 : frozen ? 0.95 : 0.34;
```

```
+      view.sprite.alpha = 1 - collapseT * 0.45;
+    }
+  }
+
+  private renderFlybys(flybys: readonly FlybyState[]): void {
+    for (let index = 0; index < this.flybyViews.length; index += 1) {
+      const flyby = flybys[index];
+      const view = this.flybyViews[index];
+      if (!flyby?.active) {
+        view.sprite.visible = false;
+        view.streak.visible = false;
+        continue;
+      }
+      view.sprite.visible = true;
+      view.streak.visible = true;
+      view.sprite.position.set(flyby.x, flyby.y);
+      view.sprite.scale.set(flyby.radius / 24);
+      view.streak.clear();
+      view.streak.moveTo(flyby.x - flyby.vx * 0.08, flyby.y - flyby.vy * 0.08);
+      view.streak.lineTo(flyby.x, flyby.y);
+      view.streak.stroke({ color: 0xfff0a3, alpha: 0.55, width: 8 });
+    }
+  }
+
+  private renderHud(snapshot: SimulationSnapshot): void {
+    const width = (WORLD_WIDTH - 172) * clamp(snapshot.energy / MAX_ENERGY, 0, 1);
+    const barColor = snapshot.phase >= 4 ? 0xff5d73 : snapshot.phase >= 3 ? 0xffc857 : 0x67f4ff;
+    this.energyBarFill.clear();
+    this.energyBarFill.roundRect(86, WORLD_HEIGHT - 124, width, 22, 6).fill({ color: barColor, alpha: 0.95 });
+    this.energyBarFill.roundRect(86, WORLD_HEIGHT - 124, width, 22, 6).fill({ color: 0xffffff, alpha: 0.14 });
+    this.scoreText.text = snapshot.score.toString();
+    this.metaText.text = `${formatTime(this.sim.getReplayPayload().survivalMs)}  PHASE ${snapshot.phase}  x${snapshot.streak}`;
+    this.endText.visible = snapshot.ended;
+    this.endText.text = snapshot.ended
+      ? `GALAXY COLLAPSED\n${snapshot.score}  •  ${formatTime(this.sim.getReplayPayload().survivalMs)}`
+      : '';
+  }
+
+  private renderCollapse(snapshot: SimulationSnapshot): void {
+    this.collapseLayer.clear();
+    if (!snapshot.ended) {
+      return;
+    }
+    const t = snapshot.collapseT;
+    this.world.scale.set(this.scale * (1 - t * 0.06));
+    this.world.position.set(this.offsetX + WORLD_WIDTH * this.scale * t * 0.03, this.offsetY + WORLD_HEIGHT * this.scale * t * 0.025);
+    this.collapseLayer.rect(0, 0, WORLD_WIDTH, WORLD_HEIGHT).fill({ color: 0x000000, alpha: clamp((t - 0.42) / 0.58, 0, 1) });
+  }
+
+  private readonly resize = (): void => {
+    const rect = this.root.getBoundingClientRect();
+    this.scale = Math.min(rect.width / WORLD_WIDTH, rect.height / WORLD_HEIGHT);
+    const viewWidth = WORLD_WIDTH * this.scale;
+    const viewHeight = WORLD_HEIGHT * this.scale;
+    this.offsetX = (rect.width - viewWidth) / 2;
+    this.offsetY = (rect.height - viewHeight) / 2;
+    this.world.position.set(this.offsetX, this.offsetY);
+    this.world.scale.set(this.scale);
+  };
+
+  private screenToWorld(clientX: number, clientY: number): WorldPoint {
+    const rect = this.app.canvas.getBoundingClientRect();
+    return {
+      x: clamp((clientX - rect.left - this.offsetX) / this.scale, 0, WORLD_WIDTH),
+      y: clamp((clientY - rect.top - this.offsetY) / this.scale, 0, WORLD_HEIGHT)
+    };
+  }
+
+  private sampleFrame(label: string): void {
+    try {
+      const dataUrl = this.app.canvas.toDataURL('image/png', 0.72);
+      this.frameSamples.push({ dataUrl, label });
+      if (this.frameSamples.length > 3) {
+        this.frameSamples.shift();
+      }
+    } catch {
+      this.frameSamples = this.makeFallbackFrames();
+    }
+  }
+
+  private makeFallbackFrames(): PosterFrame[] {
+    const snapshot = this.sim.getSnapshot();
+    return [0, 1, 2].map((index) => ({
+      dataUrl: this.makeMockFrame(snapshot, index),
+      label: `mock-${index}`
+    }));
+  }
+
+  private makeMockFrame(snapshot: SimulationSnapshot, index: number): string {
+    const canvas = document.createElement('canvas');
+    canvas.width = 540;
+    canvas.height = 960;
+    const context = canvas.getContext('2d');
+    if (!context) {
+      return '';
+    }
+    context.fillStyle = '#03040a';
+    context.fillRect(0, 0, canvas.width, canvas.height);
+    context.fillStyle = index === 0 ? '#143c5c' : index === 1 ? '#3c285f' : '#501421';
+    context.beginPath();
+    context.arc(270, 410, 180 + snapshot.phase * 24, 0, Math.PI * 2);
+    context.fill();
```

```
+    context.fillStyle = '#000000';
+    context.beginPath();
+    context.arc(270, 410, 62 + snapshot.phase * 15, 0, Math.PI * 2);
+    context.fill();
+    context.fillStyle = '#f7fbff';
+    context.font = '700 54px system-ui';
+    context.fillText(String(snapshot.score), 52, 830);
+    return canvas.toDataURL('image/png', 0.8);
+  }
+}
diff --git a/src/game/Simulation.ts b/src/game/Simulation.ts
new file mode 100644
index 0000000..125f43b
--- /dev/null
+++ b/src/game/Simulation.ts
@@ -0,0 +1,553 @@
+import {
+  BLACK_HOLE_X,
+  BLACK_HOLE_Y,
+  FLYBY_POOL_SIZE,
+  INITIAL_ENERGY,
+  MAX_ENERGY,
+  ORB_POOL_SIZE,
+  WORLD_HEIGHT,
+  WORLD_WIDTH
+} from './constants';
+import { clamp, distanceSquared, distanceToSegmentSquared } from './math';
+import { createSeededRandom, type RandomSource } from './rng';
+import type {
+  FlybyState,
+  GamePhase,
+  OrbState,
+  PhaseTransition,
+  ReplayPayload,
+  ShadowArmState,
+  SimulationSnapshot,
+  SwipeEvent,
+  TapEvent
+} from './types';
+
+const ORB_CAPTURE_RADIUS = 82;
+const FLYBY_CAPTURE_RADIUS = 58;
+const EVENT_HORIZON_RADIUS = 98;
+const ARM_LENGTH = 760;
+const ARM_HIT_WIDTH = 46;
+
+export interface SimulationOptions {
+  seed: string;
+  startedAt: number;
+}
+
+export interface TimedInput {
+  kind: 'tap' | 'swipe';
+  t: number;
+  x: number;
+  y: number;
+  x2?: number;
+  y2?: number;
+}
+
+export class Simulation {
+  readonly seed: string;
+  readonly startedAt: number;
+  private rng: RandomSource;
+  private nextOrbId = 1;
+  private nextFlybyId = 1;
+  private orbSpawnMs = 900;
+  private flybySpawnMs = 2600;
+  private readonly tapEvents: TapEvent[] = [];
+  private readonly swipeEvents: SwipeEvent[] = [];
+  private readonly phaseTransitions: PhaseTransition[] = [];
+  private readonly orbs: OrbState[] = [];
+  private readonly flybys: FlybyState[] = [];
+  private readonly shadowArms: ShadowArmState[] = [];
+  private timeMs = 0;
+  private score = 0;
+  private energy = INITIAL_ENERGY;
+  private energyCaptured = 0;
+  private streak = 0;
+  private maxStreak = 0;
+  private phase: GamePhase = 1;
+  private ended = false;
+  private collapseMs = 0;
+
+  constructor(options: SimulationOptions) {
+    this.seed = options.seed;
+    this.startedAt = options.startedAt;
+    this.rng = createSeededRandom(options.seed);
+    this.initPools();
+    this.phaseTransitions.push({ t: 0, phase: 1, energy: this.energy });
+  }
+
+  reset(): void {
+    this.rng = createSeededRandom(this.seed);
+    this.nextOrbId = 1;
+    this.nextFlybyId = 1;
+    this.orbSpawnMs = 900;
+    this.flybySpawnMs = 2600;
+    this.tapEvents.length = 0;
+    this.swipeEvents.length = 0;
+    this.phaseTransitions.length = 0;
+    this.timeMs = 0;
```

```
+    this.score = 0;
+    this.energy = INITIAL_ENERGY;
+    this.energyCaptured = 0;
+    this.streak = 0;
+    this.maxStreak = 0;
+    this.phase = 1;
+    this.ended = false;
+    this.collapseMs = 0;
+    this.initPools();
+    this.phaseTransitions.push({ t: 0, phase: 1, energy: this.energy });
+  }
+
+  step(dtMs: number): void {
+    const dt = dtMs / 1000;
+    if (this.ended) {
+      this.timeMs += dtMs;
+      this.collapseMs = Math.min(this.collapseMs + dtMs, 2200);
+      return;
+    }
+
+    this.timeMs += dtMs;
+    this.energy = clamp(this.energy - dt * (0.28 + this.phase * 0.05), 0, MAX_ENERGY);
+    this.spawnOrbs(dtMs);
+    this.spawnFlybys(dtMs);
+    this.updateShadowArms(dtMs);
+    this.updateOrbs(dtMs, dt);
+    this.updateFlybys(dtMs, dt);
+    this.updatePhase();
+
+    if (this.energy <= 0) {
+      this.endRun();
+    }
+  }
+
+  applyTap(x: number, y: number): TapEvent {
+    let target: TapEvent['target'] = 'empty';
+    const flyby = this.findFlyby(x, y);
+    if (flyby) {
+      this.collectFlyby(flyby);
+      target = 'flyby';
+    } else {
+      const orb = this.findOrb(x, y, ORB_CAPTURE_RADIUS);
+      if (orb) {
+        orb.frozenMs = Math.max(orb.frozenMs, 520);
+        this.score += 8 + this.phase;
+        target = 'orb';
+      } else if (this.stunArm(x, y)) {
+        target = 'arm';
+      }
+    }
+
+    const event: TapEvent = { t: Math.round(this.timeMs), x: Math.round(x), y: Math.round(y), target };
+    this.tapEvents.push(event);
+    return event;
+  }
+
+  applySwipe(x1: number, y1: number, x2: number, y2: number): SwipeEvent {
+    const orb = this.findOrbOnSegment(x1, y1, x2, y2);
+    const target: SwipeEvent['target'] = orb ? 'orb' : 'empty';
+    if (orb) {
+      this.captureOrb(orb);
+    }
+    const event: SwipeEvent = {
+      t: Math.round(this.timeMs),
+      x1: Math.round(x1),
+      y1: Math.round(y1),
+      x2: Math.round(x2),
+      y2: Math.round(y2),
+      target
+    };
+    this.swipeEvents.push(event);
+    return event;
+  }
+
+  forceEnd(): void {
+    this.energy = 0;
+    this.endRun();
+  }
+
+  getSnapshot(): SimulationSnapshot {
+    return {
+      timeMs: Math.round(this.timeMs),
+      score: this.score,
+      energy: this.energy,
+      energyCaptured: this.energyCaptured,
+      streak: this.streak,
+      maxStreak: this.maxStreak,
+      phase: this.phase,
+      ended: this.ended,
+      collapseT: clamp(this.collapseMs / 2200, 0, 1),
+      orbs: this.orbs,
+      flybys: this.flybys,
+      shadowArms: this.shadowArms
+    };
+  }
+
+  getReplayPayload(): ReplayPayload {
+    return {
+      version: 1,
+      seed: this.seed,
+      startedAt: this.startedAt,
+      survivalMs: Math.round(this.ended ? this.timeMs - this.collapseMs : this.timeMs),
```

```
+      score: this.score,
+      energyCaptured: this.energyCaptured,
+      maxStreak: this.maxStreak,
+      tapEvents: this.tapEvents.map((event) => ({ ...event })),
+      swipeEvents: this.swipeEvents.map((event) => ({ ...event })),
+      phaseTransitions: this.phaseTransitions.map((event) => ({ ...event }))
+    };
+  }
+
+  runInputs(inputs: TimedInput[], untilMs: number): ReplayPayload {
+    let inputIndex = 0;
+    const ordered = [...inputs].sort((a, b) => a.t - b.t);
+    while (this.timeMs < untilMs && !this.ended) {
+      while (inputIndex < ordered.length && ordered[inputIndex].t <= this.timeMs) {
+        const input = ordered[inputIndex];
+        if (input.kind === 'tap') {
+          this.applyTap(input.x, input.y);
+        } else {
+          this.applySwipe(input.x, input.y, input.x2 ?? input.x, input.y2 ?? input.y);
+        }
+        inputIndex += 1;
+      }
+      this.step(1000 / 60);
+    }
+    return this.getReplayPayload();
+  }
+
+  private initPools(): void {
+    this.orbs.length = 0;
+    this.flybys.length = 0;
+    this.shadowArms.length = 0;
+    for (let index = 0; index < ORB_POOL_SIZE; index += 1) {
+      this.orbs.push(this.makeInactiveOrb(index));
+    }
+    for (let index = 0; index < FLYBY_POOL_SIZE; index += 1) {
+      this.flybys.push(this.makeInactiveFlyby(index));
+    }
+    for (let index = 0; index < 3; index += 1) {
+      this.shadowArms.push({
+        angle: index * ((Math.PI * 2) / 3),
+        stunMs: 0,
+        intensity: 0
+      });
+    }
+  }
+
+  private makeInactiveOrb(index: number): OrbState {
+    return {
+      active: false,
+      captured: false,
+      id: -index,
+      x: 0,
+      y: 0,
+      vx: 0,
+      vy: 0,
+      radius: 26,
+      ageMs: 0,
+      frozenMs: 0,
+      energy: 1,
+      tetherPhase: 0
+    };
+  }
+
+  private makeInactiveFlyby(index: number): FlybyState {
+    return {
+      active: false,
+      id: -index,
+      x: 0,
+      y: 0,
+      vx: 0,
+      vy: 0,
+      radius: 24,
+      ageMs: 0
+    };
+  }
+
+  private spawnOrbs(dtMs: number): void {
+    this.orbSpawnMs -= dtMs;
+    if (this.orbSpawnMs > 0) {
+      return;
+    }
+    this.orbSpawnMs = clamp(1020 - this.phase * 120 - this.timeMs * 0.006, 430, 1100);
+    const orb = this.orbs.find((candidate) => !candidate.active);
+    if (!orb) {
+      return;
+    }
+
+    const edge = Math.floor(this.rng() * 4);
+    const margin = 96;
+    let x = margin + this.rng() * (WORLD_WIDTH - margin * 2);
+    let y = margin + this.rng() * (WORLD_HEIGHT - margin * 2);
+    if (edge === 0) {
+      y = margin;
+    } else if (edge === 1) {
+      x = WORLD_WIDTH - margin;
+    } else if (edge === 2) {
+      y = WORLD_HEIGHT - margin * 1.9;
+    } else {
+      x = margin;
+    }
+    const angle = Math.atan2(y - BLACK_HOLE_Y, x - BLACK_HOLE_X);
+    const tangentSign = this.rng() > 0.5 ? 1 : -1;
```

```
+    const tangent = angle + (Math.PI / 2) * tangentSign + (this.rng() - 0.5) * 0.45;
+    const speed = 76 + this.rng() * 48 + this.phase * 12;
+    const inwardX = BLACK_HOLE_X - x;
+    const inwardY = BLACK_HOLE_Y - y;
+    const inwardLength = Math.max(1, Math.hypot(inwardX, inwardY));
+
+    orb.active = true;
+    orb.captured = false;
+    orb.id = this.nextOrbId;
+    this.nextOrbId += 1;
+    orb.x = x;
+    orb.y = y;
+    orb.vx = Math.cos(tangent) * speed + (inwardX / inwardLength) * 42;
+    orb.vy = Math.sin(tangent) * speed + (inwardY / inwardLength) * 42;
+    orb.radius = 24 + this.rng() * 13;
+    orb.ageMs = 0;
+    orb.frozenMs = 0;
+    orb.energy = 2 + this.rng() * 2.8;
+    orb.tetherPhase = this.rng() * Math.PI * 2;
+  }
+
+  private spawnFlybys(dtMs: number): void {
+    this.flybySpawnMs -= dtMs;
+    if (this.flybySpawnMs > 0) {
+      return;
+    }
+    this.flybySpawnMs = 4700 + this.rng() * 1900 - this.phase * 320;
+    const flyby = this.flybys.find((candidate) => !candidate.active);
+    if (!flyby) {
+      return;
+    }
+    const fromLeft = this.rng() > 0.5;
+    const y = 240 + this.rng() * (WORLD_HEIGHT - 620);
+    const speed = 760 + this.rng() * 240;
+    flyby.active = true;
+    flyby.id = this.nextFlybyId;
+    this.nextFlybyId += 1;
+    flyby.x = fromLeft ? -80 : WORLD_WIDTH + 80;
+    flyby.y = y;
+    flyby.vx = fromLeft ? speed : -speed;
+    flyby.vy = (this.rng() - 0.5) * 110;
+    flyby.radius = 22 + this.rng() * 9;
+    flyby.ageMs = 0;
+  }
+
+  private updateShadowArms(dtMs: number): void {
+    const dt = dtMs / 1000;
+    for (let index = 0; index < this.shadowArms.length; index += 1) {
+      const arm = this.shadowArms[index];
+      arm.angle += dt * (0.12 + this.phase * 0.045) * (index % 2 === 0 ? 1 : -1);
+      arm.stunMs = Math.max(0, arm.stunMs - dtMs);
+      arm.intensity = arm.stunMs > 0 ? 0.25 : clamp((this.phase - 1) / 3, 0, 1);
+    }
+  }
+
+  private updateOrbs(dtMs: number, dt: number): void {
+    for (const orb of this.orbs) {
+      if (!orb.active) {
+        continue;
+      }
+
+      orb.ageMs += dtMs;
+      if (orb.captured) {
+        const captureT = clamp(dt * 8, 0, 1);
+        orb.x += (BLACK_HOLE_X - orb.x) * captureT;
+        orb.y += (WORLD_HEIGHT - 112 - orb.y) * captureT;
+        orb.radius *= 1 + dt * 0.8;
+        if (orb.ageMs > 460) {
+          orb.active = false;
+        }
+        continue;
+      }
+
+      if (orb.frozenMs > 0) {
+        orb.frozenMs = Math.max(0, orb.frozenMs - dtMs);
+      } else {
+        const dx = BLACK_HOLE_X - orb.x;
+        const dy = BLACK_HOLE_Y - orb.y;
+        const radiusSq = dx * dx + dy * dy;
+        const radius = Math.max(48, Math.sqrt(radiusSq));
+        const gravity = (th
```